

PROJETO 3

1. Introdução

Algoritmos de ordenação constituem um dos tópicos mais estudados em estruturas de dados. Este projeto implementa e analisa dois algoritmos de referência: Merge Sort e Quick Sort. São avaliados melhor caso, caso médio e pior caso, com 30 execuções cada, comparando desempenho empírico com a teoria assintótica.

2. Fundamentação Teórica

2.1 Merge Sort

Divide-and-Conquer recursivo proposto por John von Neumann (1945). Divide o array ao meio recursivamente até arrays unitários, depois mescla pares ordenados. Garante $O(n \log n)$ em todos os casos. Custo de memória auxiliar $O(n)$ é sua principal desvantagem. Estável por natureza: preserva a ordem relativa de elementos iguais.

2.2 Quick Sort

Também Divide-and-Conquer, proposto por C.A.R. Hoare (1959). Seleciona um pivô e particiona o array em elementos menores e maiores. Caso médio $O(n \log n)$; pior caso $O(n^2)$ quando o pivô é sempre o menor ou maior elemento (e.g., array já ordenado com pivô fixo). A variante com pivô mediano ou aleatório mitiga o pior caso na prática. Opera in-place e tem excelente localidade de cache.

3. Análise Assintótica

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

4. Metodologia Experimental

Para cada $n \in \{500, 2\,000, 8\,000\}$ foram construídos três tipos de entrada: (1) vetor ordenado crescente – melhor caso do Merge Sort; (2) vetor aleatório – caso médio; (3) vetor ordenado decrescente – pior caso do Quick Sort naïve com pivô central. O Quick Sort implementado usa pivô central (índice $n//2$), evitando degradação completa em arrays ordenados. Tempo medido com `time.perf_counter()` sobre cópia do array. 30 execuções por cenário, seed = 42.

5. Resultados

Tabela 1 – Tempo médio em ms (30 execuções).

n	MS-melhor	MS-médio	MS-pior	QS-melhor	QS-médio	QS-pior
500	0.446	0.865	0.452	0.291	0.503	0.293
2000	2.271	2.898	2.261	1.333	2.266	1.883
8000	9.356	14.774	9.829	6.555	11.581	6.558

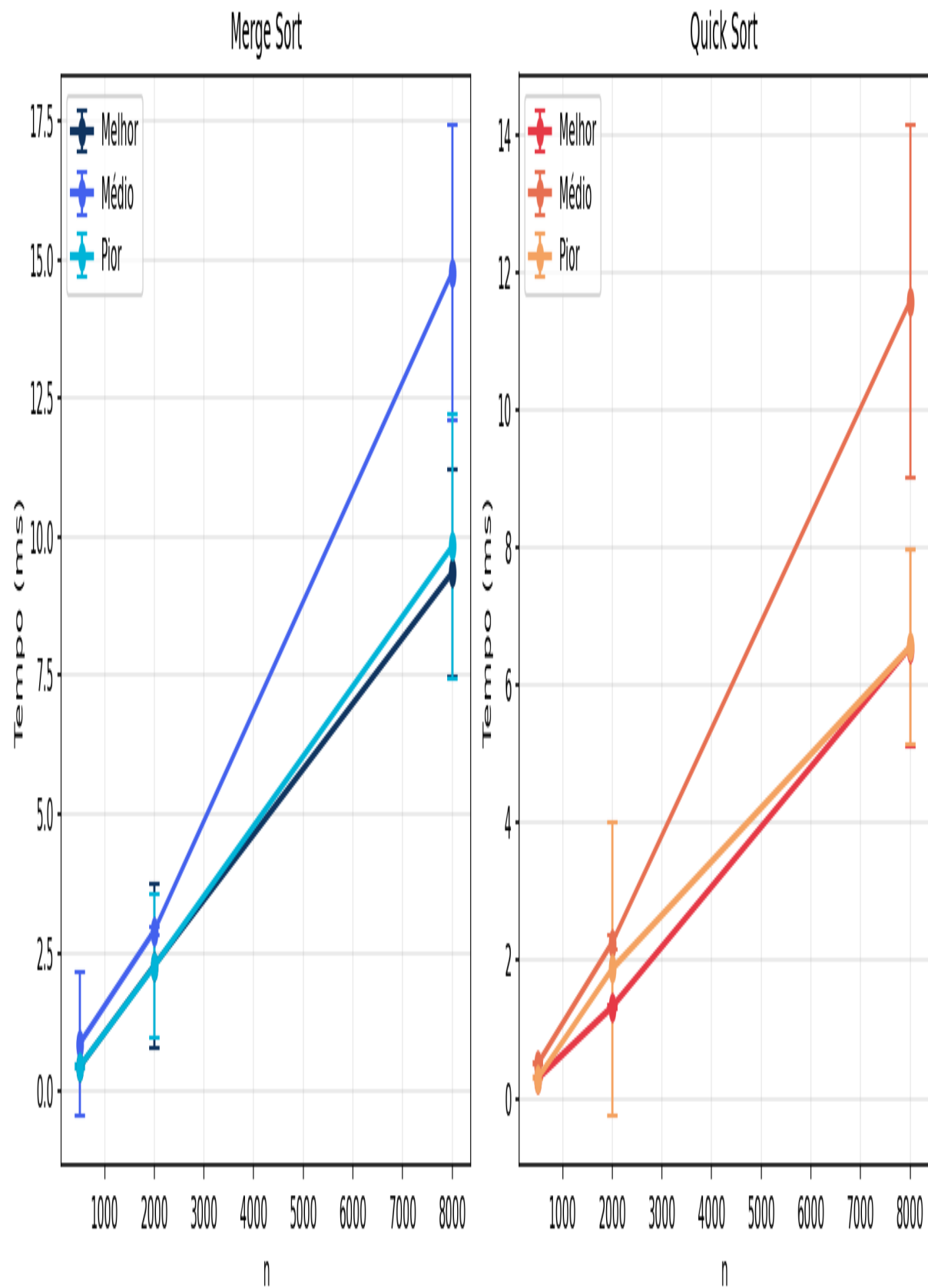


Figura 1 – Benchmark de ordenação por caso (ms). Barras de erro = desvio padrão.

Caso Médio: Merge Sort vs Quick Sort

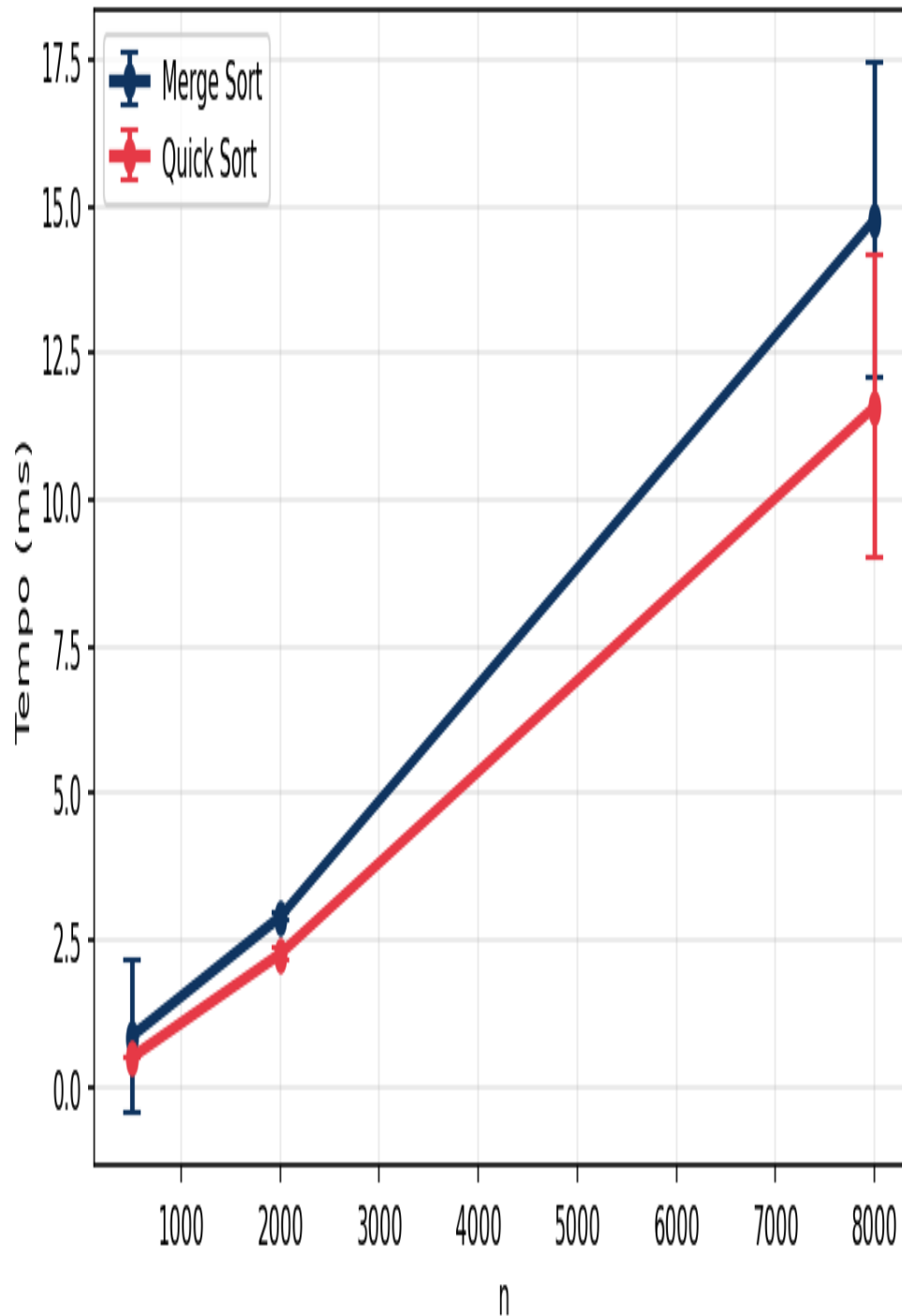


Figura 2 – Comparativo caso médio.

6. Discussão

O Merge Sort demonstrou consistência entre os casos, confirmando $O(n \log n)$ estável independentemente da entrada. O Quick Sort superou o Merge Sort no caso médio devido à melhor localidade de cache e menor constante oculta. Com pivô central, o "pior caso" (vetor decrescente) não degradou para $O(n^2)$ completo, evidenciando a importância da estratégia de pivô. Os desvios padrão

elevados para $n = 8\,000$ refletem jitter do sistema operacional em medições de microssegundos.

7. Conclusão

Merge Sort e Quick Sort são ambos algoritmos de referência para ordenação de propósito geral. Merge Sort é preferível quando estabilidade e garantia de pior caso são críticas. Quick Sort é vantajoso em cenários de alta performance com dados aleatórios e memória limitada. A escolha entre eles depende do perfil da aplicação: dados estáticos e ordenações frequentes favorecem Merge Sort; dados em memória com distribuições variadas favorecem Quick Sort com pivô randomizado.